Chapter 1

Introduction

1.1. Background and motivation

Road traffic crashes remain a leading cause of death and serious injury worldwide [1], and the overwhelming majority are attributed to human error [2], as cited by Medina et al. [3]. Autonomous driving promises to reduce severe and fatal accidents (e.g. severe lane-departure accidents [4]), improve traffic flow [5], and reduce emissions [6]. Before such systems can be trusted on public roads, however, they must handle difficult situations, where small mistakes compound quickly.

Autonomous racing is a demanding testbed for exactly these situations. The high speeds, tight control margins, and short time available to recover from bad decisions make it a proxy for developing and evaluating autonomous driving algorithms.

Broadly, autonomous driving control architectures fall into two paradigms: modular pipelines and end-to-end learning. Modular pipelines decompose the driving task into distinct sub-problems, such as perception, planning, and control. While this separation provides interpretability and simplifies debugging, it often demands extensive domain expertise and manual tuning. Conversely, end-to-end learning directly maps raw sensor inputs (e.g., LiDAR or camera frames) to control outputs (e.g., speed and steering angle). Although this approach bypasses the need for hand-engineered intermediate representations, it typically requires vast amounts of data and suffers from poor interpretability, making failures difficult to diagnose [7].

Auxiliary learning presents a compelling method to address these shortcomings. By training a model to perform secondary tasks alongside its primary control objective, the network is encouraged to learn richer, more structured intermediate representations [8] while improving interpretability [7]. This paradigm is particularly useful for end-to-end racing in simulation. Because the simulator has access to privileged, ground-truth state information - such as the vehicle's true velocity, tire slip, and upcoming track

geometry - these metrics can be used as auxiliary supervisory signals during training, even though they remain inaccessible to the deployed policy during inference.

1.2. Problem statement

An end-to-end reinforcement learning (RL) agent that drives from LiDAR alone has to learn representations of privileged information that it cannot directly observe in the real world: its own velocity and slip, the curvature of the track ahead, and, in Head-to-Head (H2H) racing, the opponent's location. Early in training, the agent has not yet learned to drive well enough to explore the track effectively, so the reward signal that would teach it these representations is sparse and noisy. This can lead to slow convergence.

Privileged information is, however, available in simulation. This report investigates whether *auxiliary losses* that predict this privileged information from the policy's shared representation can improve the learning of a LiDAR-only end-to-end racing agent.

1.3. Objectives

The objectives of this project are to:

(a) select an appropriate simulation environment for autonomous racing;
(b) implement a baseline end-to-end RL agent that drives from LiDAR alone;
(c) gain an understanding of what representations the baseline agent natively encodes;
(d) implement auxiliary learning to provide additional supervisory signals to the agent during training; and
(e) evaluate the performance of the auxiliary learning agent against the baseline agent.

Meeting these objectives answers the research question: (a) and (b) provide the experimental platform and the baseline, (c) establishes what auxiliary learning must improve on, and (d) and (e) test the proposed method directly.

1.4. Summary of work

To answer the research question, the project was executed in three phases: simulation development, baseline evaluation, and auxiliary learning experimentation.

First, a custom 3D racing simulator was built in Unity. It uses PhysX-based vehicle dynamics from the RoboRacer platform and features a procedural track generator. It

was benchmarked against the 2D F1Tenth gym and, despite offering high-fidelity physics (including weight transfer and non-linear tyre slip), it ran only 19% slower, justifying its use for training.

Second, a baseline end-to-end RL agent was trained using Proximal Policy Optimisation (PPO) and a dense progress-based reward. The agent's memoryless feedforward policy drove using only a stack of 50 LiDAR scans (the previous 2s) and the current throttle and steering angle. To understand what representations this baseline naturally encoded, linear regression probes were fitted to the activations of its final shared layer. Even at random initialisation, the network natively captured certain targets: lateral and angular velocities were linearly decodable with mean out-of-sample R^2 scores of 92% and 85% respectively.

Finally, auxiliary heads were added to the network to predict privileged state information (forward, lateral, and angular velocity; per-wheel slip; and upcoming track curvature) during training. The models were trained with a weighted auxiliary loss, yielding these primary findings:

- Predicting close track curvature ($\leq 3m$ ahead) forced the network to encode measurably more spatial information (an absolute increase of between +51 to +58 percentage points in R^2 scores for the close track curvature probes, and +28 and +10 for the 3.5m- and 4.0m-ahead probes which are outside of the auxiliary supervision range). The agent also drove measurably faster (+12.5%) and had fewer collisions per metre of progress (-37.67%) than the baseline.
- Predicting far-ahead track curvature (6m to 8m ahead) was not learned well by the model, and resulted in degraded performance (-24.6% average velocity) and an elevated crash rate per metre progress (up to +265%).
- Predicting the forward slip of either the front or rear wheels decreased the agent's crash rate per metre of progress for both back and front wheels (-42.6% and - 48.5% respectively), but only forward wheel slip prediction improved the agent's velocity (+7.0%), though both only when the auxiliary loss had a low weight (0.1)

1.5. Scope

This report is limited to simulation; no sim-to-real transfer to a physical RoboRacer was attempted. The agent drives from LiDAR and its current steering and throttle only, and uses a memoryless feedforward policy with a stacked observation history rather than a recurrent architecture. Learning is done with PPO alone, and other RL algorithms were

not compared. Only auxiliary *losses* are studied; auxiliary rewards and reward shaping are discussed as background but not evaluated. Representations are analysed with linear probes only, so non-linearly encoded information is not measured. Only single-agent racing is studied, so opponent information is not considered.

1.6. Roadmap

[Fill in]

Chapter 2

Background and Related Literature

2.1. Previous Approaches

Evans et al. [9] present a survey breaking down the autonomous racing literature in RoboRacer into two categories: *classical approaches* and *learning approaches*. We shall provide a brief overview of classical approaches, and then focus on learning approaches, which are more relevant to this project.

2.1.1. Classical approaches

Classical approaches generally decompose the driving task into three stages: perception, planning, and control [9]. Perception is the process of converting raw sensor inputs into a representation of the environment, such as a map of the track and the location of other vehicles. Planning uses this representation to generate a trajectory for the vehicle to follow, and control executes this trajectory by sending commands to the vehicle's actuators. These approaches tend to be more interpretable and easier to debug [10], as each component can be analysed separately [11].

2.1.2. Learning approaches

Learning approaches, which have recently shown promise in the field of autonomous driving in simulation [12], [13] as well as in deployment on a physical vehicle [14], are often end-to-end, mapping raw sensor inputs directly to control outputs [11]. This can be done using supervised learning (such as behaviour cloning [15]), where the model is trained on a dataset of expert demonstrations, or using reinforcement learning [16], where the model learns by interacting with the environment and receiving feedback in the form of rewards.

Learning approaches have the potential to learn complex behaviours that are difficult to hand-engineer, and can adapt to changing environments [17]. However, they often require large amounts of data and computational resources [18], and can be less interpretable than classical approaches: deep end-to-end learning approaches are often regarded as "black boxes" [19], which may be undesirable in safety-critical applications [20]. In RoboRacer, learning approaches have shown promise, but were unable to outperform classical approaches in benchmarks [9], and recently the majority of 27th RoboRacer Autonomous Racing Competition entries were classical approaches [21], despite learning approaches generalising better to unseen tracks [17].

2.2. Learning Methods Using Privileged Information

During training in simulation, information that is not available to the agent at inference time, is available. This information is referred to as *privileged information*, and is usually unused. There are, however, several methods that use privileged information to improve the performance of an agent.

Common ways to use privileged information include as inputs to training-only components (e.g., an asymmetric critic), as inputs to a teacher for distillation, and as auxiliary rewards or losses. We provide these examples to illustrate how other methods use privileged information, and to motivate the use of auxiliary losses as an alternative method in this project.

2.2.1. Asymetric Actor-critic Methods

Asymetric actor-critic methods [22] are a variant of actor-critic methods that train two distinct models: one for the policy, which chooses the agent's actions and only receives the agent's observations as input, and one for the critic, which receives additional privileged informaton as input. The critic is used to estimate the value of each state, which is then used to update the agent to choose actions that lead it into more valuable states. Vasco et al. [23] show that an asymmetric critic can achieve better-than-human performance of a vision-based RL agent in a simulated autonomous racing environment, and perform better than a baseline symmetric actor-critic agent.

2.2.2. Teacher-student distillation

Teacher-student distillation [24] involves training a teacher model with access to privileged information, and then using the teacher's outputs to train a student model that

does not have access to this information. The student model is trained to mimic the teacher's behaviour, effectively learning from the teacher's understanding of its privileged knowledge. Xu et al. [25] applied this method to LiDAR-based autonomous racing on the RoboRacer platform, and found that the student model was able to achieve better performance than a baseline model trained without a teacher.

2.2.3. Auxiliary learning

Auxiliary learning is a method that falls under the umbrella of multi-task learning, where a model is trained to perform multiple tasks simultaneously. Specifically, in auxiliary learning, the model's performance on the auxiliary task is not an objective in itself, but the model's performance on the auxiliary task is used to improve its performance on the primary task. This occurs because the auxiliary task provides additional information that can be used to improve the model's understanding of the primary task, which can lead to better generalisation and faster learning [26].

2.2.3.1. Auxiliary Rewards

When privileged information is available, it can be used to construct auxiliary rewards that guide the agent toward desirable behaviors by evaluating performance against sub-goals. For example, in autonomous racing, privileged knowledge of track layout or opponent positions can generate bonuses for staying centered or avoiding near-misses.

While auxiliary rewards act as a form of reward shaping - supplying denser feedback to accelerate learning - there is a crucial distinction between auxiliary rewards and reward shaping. Traditional potential-based shaping, where the agent is, for example, rewarded for moving towards a target and penalised for moving away from it, guarantees that the underlying task and optimal policy remain unchanged [27]. Auxiliary rewards, however, act as heuristic bonuses that are rarely potential-based. Consequently, they risk altering the task being solved, which can cause the agent to converge on suboptimal policies for the original objective [27].

2.2.3.2. Auxiliary losses

Incorporation of auxiliary losses alongside the primary loss has emerged as a promising technique for improving the performance, interpretability, and training time of end-to-end learning approaches. It does so in three ways: it reduces the variance of the learning signal, it allows privileged information to be used even when it is unavailable at inference time, and it improves representation quality [28], [29].

Examples of auxiliary learning with privliged information prediction include Lample et al. [30] who demonstrated outstanding performance in first-person shooter games

by training an agent to predict the presence of enemies in the agent's field of view (FOV), and Mirowski et al. [31] who found auxiliary tasks for depth prediction and loop closure detection improved navigation in 3D mazes, even compared to agents trained with depth information directly as input.

In the field of autonomous driving, Chitta et al. [32] applied auxiliary losses for depth prediction, semantic segmentation, and vehicle detection, achieving state-of-the-art performance on the CARLA benchmark, and significantly decreasing the crash rate. Chen et al. [7] use auxiliary losses for birds-eye-view prediction in urban environments, improving both interpretability and

2.3. Evaluation of existing approaches

End-to-end RL agents on the RoboRacer platform generalise well to unseen tracks but have yet to match classical pipelines, and their opaque representations make failures hard to diagnose. In simulation, privileged state such as velocity, tyre slip and upcoming track geometry is available but unused at deployment. Existing ways of exploiting it each leave a limitation: An asymmetric critic improves value estimation but doesn't directly shape the policy's own representation; teacher-student distillation is complex since it requires a separate teacher and a two-stage pipeline; and auxiliary rewards alter the optimisation objective and must still pass through the high-variance policy gradient. Auxiliary losses avoid all three: they leave the RL objective unchanged, supply a lower-variance supervised gradient directly to the shared representation, and are simple to implement. They have improved performance in first-person games, 3D navigation and urban driving. However, to our knowledge, they have not been studied for LiDAR-only racing, nor has it been examined which privileged targets help, or how they change what the policy's representation encodes. This project addresses that gap.

Chapter 3

System overview

The system trains an end-to-end RL agent to drive a simulated RoboRacer vehicle from as input LiDAR. It runs entirely in simulation and has two subsystems, shown in Figure 3.1: a simulator, which models the vehicle, track and physics interactions, and a Python training script, which computes rewards given the state communicated by the simulator, and updates the policy. The two communicate over gRPC through the ML-Agents toolkit [33], [34].

At each timestep t, the simulator sends a subset of the complete state, $b_t \in s_t$, to the training script. Only the essential information is sent to keep the communication overhead low. The training script then computes the observation o_t using the partial state information b_t and performs a forward pass through the policy using o_t as input, computing the next action a_t. This action is sent to the simulator, and is applied to the vehicle, stepping the environment forward to the next timestep $t + 1$.

To speed up training, multiple instances of the simulator are run in parallel, each with its own set of agent running in separate sandboxed simulation environments.

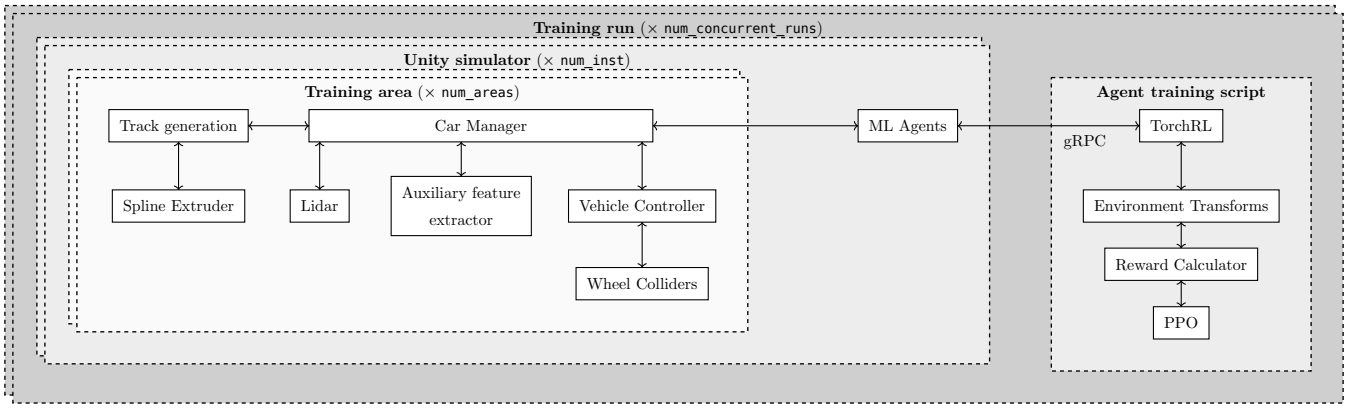


Figure 3.1: System diagram. There are `num_concurrent_runs` concurrent training runs running simultaneously, each with `num_inst` parallel simulator instances. Each simulator instance simulates `num_areas` vehicles, each with its own track. The simulator communicates with the Python training script over gRPC, and the training script updates the policy using PPO.

The following chapters shall outline the design and implementation of the two subsystems: the simulator (Chapter 4) and the training script & agent (Chapter 5).

Chapter 4

Simulator

Previous work on end-to-end driving has largely made use of the F1Tenth gym, with the F1Tenth gym being the most common simulator among methods surveyed in the survey by Evans et al. [9]. The F1Tenth gym is a Python-based 2D simulator for the F1Tenth (now RoboRacer) vehicle. This simulator is easy to use, and runs in Numba-compiled Python, but features a simplified bicycle model of the vehicle, which approximates slip dynamics using a linear function [35].

The AutoDrive simulator [36], which uses the Unity game engine [37], is a 3D simulator that models the vehicle and its environment using PhysX [38], a physics engine developed by NVIDIA. PhysX simulates rigid-body dynamics, including collisions, friction and gravity, and allows for higher-fidelity simulations, including weight transfer dynamics, and suspension deformation in Unity.

Table 4.1: Feature comparison of the Unity (PhysX) and F1Tenth gym simulators

	Unity (PhysX)	**F1Tenth gym**
Integrator	Euler	Runge-Kutta 4th order
Car model	3D, 4-wheel rigid body with suspension, weight transfer, non-linear wheel slip dynamics	2-wheel bicycle model with linearised slip dynamics
Sensors	LiDAR (max 101 rays per agent), IMU, wheel encoders[1], occupancy grids	LiDAR, IMU (excl. pitch, roll)
Track generation	No	In Beta [39]

However, the AutoDrive simulator by default does not provide various features that are important for end-to-end training in our auxiliary learning setup. For example, it is missing track generation & privileged information reporting. Therefore, a custom simulator was implemented in Unity using components from the AutoDrive simulator.

[1]The IMU and wheel encoders are modules available in the AutoDrive simulator, not native Unity

Specifically, the `VehicleController` and `IMU` components were not re-implemented, but instead used directly from the AutoDrive simulator.

4.1. Simulator performance comparison

In simulation-based training, throughput is critical because agents typically require a very large number of timesteps to learn. Two timescales are distinguished: the control timestep, at which the agent receives an observation and outputs an action, and the simulation timestep, at which the physics engine advances the numerical physics integration. The simulation timestep is usually shorter than the control timestep, since the dynamics must be integrated at a finer resolution to remain accurate. Following the settings used by Evans et al. [40], we use a control frequency of 25 Hz and a simulation frequency of 100 Hz.

In order to ensure the custom Unity simulator is suitable for training and not just evaluation, its performance was compared to the F1Tenth gym. A rollout of 10 000 control timesteps (7 minutes and 20 seconds in simulation time) was performed in each environment on an M4 MacBook Air, and the average time per control step was measured. Both environments were run in headless mode to remove any potential overhead from the graphical user interface.

Despite the Unity simulator being significantly more complex than the F1Tenth gym, it runs at a comparable speed, using a comparable amount of CPU resources in this environment, as shown in Figure 4.1, with the Unity simulator being only 19% slower compared to the F1Tenth Gym in terms of steps per second.

Due to the relatively minor performance difference, and Unity's more realistic 3D physics, the Unity simulator was chosen for training the RL agent.

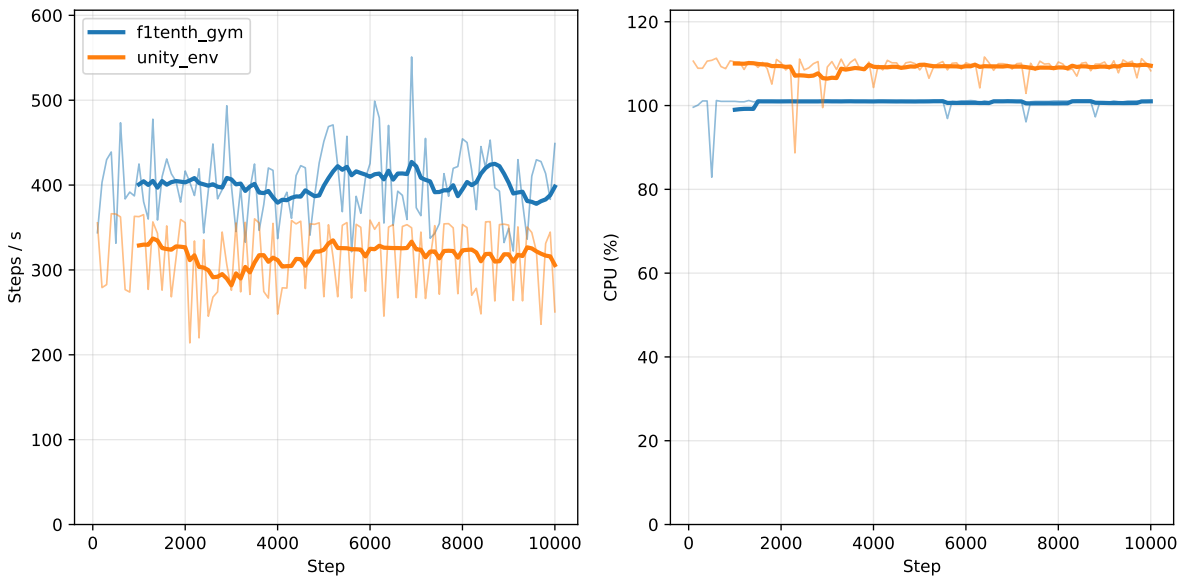


Figure 4.1: Benchmark of Unity simulator vs F1Tenth gym. In the CPU chart, a value greater than 100 indicates that the simulator used more than one CPU core. Includes child processes spawned by the simulator (e.g. in Unity's case)

4.2. Unity Vehicle Model

The vehicle is modelled as a `Rigidbody` with four wheels, each simulated by a `WheelCollider`. The two components divide the work: the `Rigidbody` represents the car as a single moving mass, and the `WheelCollider`s generate the forces that move it. Near the front of the car, a `RayPerceptionSensor` from the ML-Agents toolkit simulates a 2D LiDAR sensor with 101 rays, which is used as the agent's observation.

4.2.1. The RigidBody

A `Rigidbody` is the Unity component that gives an object physical behaviour. It stores the car's mass, centre of mass and moment of inertia. At each fixed timestep the physics engine (PhysX) integrates the forces and torques acting on it to update its linear and angular velocity and its pose. The car's body is covered by simple box colliders (Figure 4.2), which handle collisions with walls and other objects. They play no part in the tyre-road interaction, which is handled entirely by the wheels. `BoxColliders` were used to model the car's body and collisions between the wheels and walls.

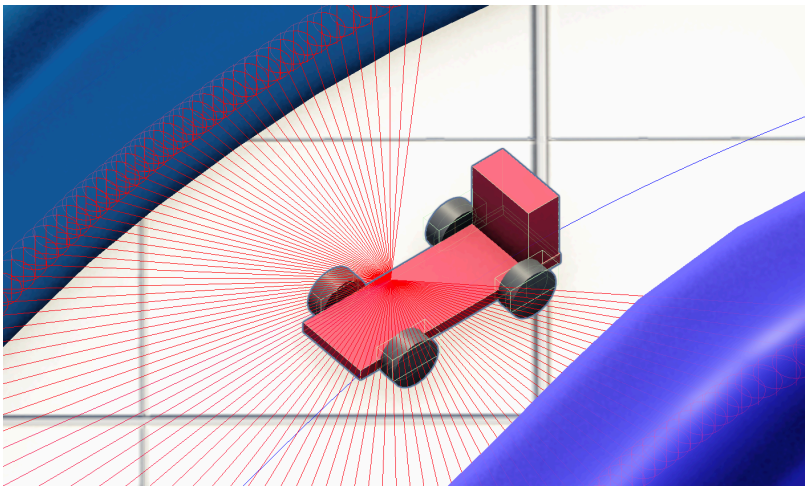
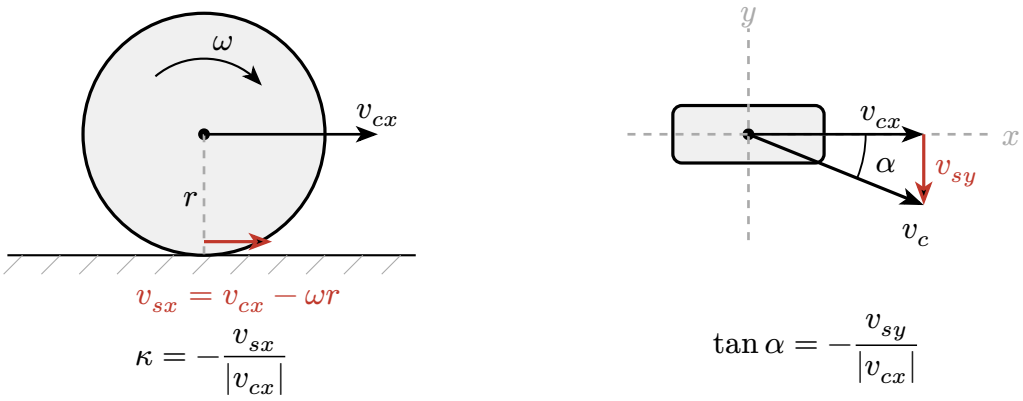


Figure 4.2: Custom Unity car's colliders: the body colliders handle collisions, and the four `WheelCollider`s handle suspension and tyre forces. The red lines show the raycasts by the `RayPerceptionSensor` that simulates the LiDAR.

4.2.2. The WheelCollider

In our custom simulator, each wheel of the RoboRacer is modelled as a `WheelCollider`, which simulates the wheel's suspension, rotation and tyre friction [41]. This kind of collider uses physics similar to that modelled by the semi-empirical "magic formula" wheel dynamics proposed by Pacejka et al. in [42], and further described in [43].

The tyre model is built on the idea of *slip*. *Longitudinal slip*, κ, is the ratio between the longitudinal slip velocity and the wheel center velocity. *Lateral slip*, $\tan(\alpha)$, is the ratio between the lateral slip velocity and the wheel center velocity. A wheel slips when it spins faster than the wheel center moves (wheelspin), turns slower than the wheel center moves (braking), or is dragged sideways (cornering). A slip value of 0 therefore indicates a freely rolling wheel, while a slip value of > 1 indicates that the point of contact has a higher velocity than the wheel center.



(a) Side view: longitudinal slip **(b)** Top view: lateral slip

Figure 4.3: Slip quantities. Longitudinal slip compares the slip velocity at the contact point with the wheel-centre velocity; lateral slip compares the sideways slip velocity with the forward velocity.

A tyre needs some slip to produce force. The `WheelCollider` computes the force from the slip using a friction curve (Figure 4.4). Force rises with slip until it reaches a peak, the *extremum*, which is the tyre's maximum grip. Beyond the peak, the tyre begins to slide and the force falls to a lower, constant value, the *asymptote*. Pushing the car harder therefore gives less force, not more, which is a regime a racing agent must learn to avoid. The curve is defined separately for the forward direction (accelerating and braking) and the sideways direction (cornering). Note that in real life, at low slip values, the rubber stretches instead of actually sliding on the road. This is modelled by the rising part of the curve at low slip values.

Figure 4.4 shows the difference between a Unity-style piecewise friction curve and a Pacejka "magic formula" curve that was fit using least-squares to approximate Unity's friction curve. The values used for our simulations are detailed in Section Appendix B and are the values calibrated by Samak and Samak [44] for the RoboRacer platform, ported to our custom Unity simulator.

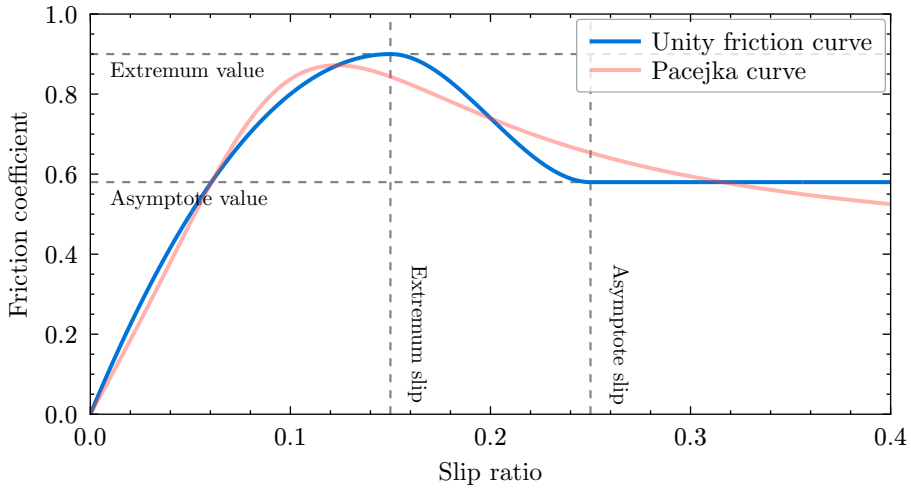


Figure 4.4: Approximated WheelCollider friction curves [41] for the RoboRacer compared to the Pacejka "magic formula" [42]

4.2.3. Steering

On the real RoboRacer, steering actuation cannot occur instantaneously, but is limited by the steering servo's maximum angular velocity. The `WheelCollider` component does not model this limitation, so we use the `VehicleController` from AutoDrive to implement a cap on steering speed of $\omega_{\max} = 315°/\,\text{sec}$. This value was chosen based on the calibration of the RoboRacer platform by Samak and Samak [44].

4.2.4. LiDAR

The vehicle is equipped with a simulated 2D LiDAR sensor, which is implemented using a `RayPerceptionSensor` from the ML-Agents toolkit. The sensor emits 101 rays in a fan pattern, covering a field of view of 270°. Each ray measures the distance to the nearest obstacle in its cast direction, and returns a normalized value between 0 and 1, where 1 indicates no obstacle within the maximum range of the sensor (20m).

4.3. The track

During training, a variety of tracks are used to prevent the policy from overfitting to any single layout. Training on diverse environments has been shown to improve generalisation to unseen environments [45], and is closely related to domain randomisation, where simulator parameters are randomised to produce more robust policies [46], [47].

4.3.1. Centerline generation

To this end, we implemented a procedural track generator in C# in Unity. The idea is a *mean-reverting random walk*: a polyline of N equal-length segments whose heading wanders randomly but is continually pulled back towards the heading of a *reference*

circle of radius R. The result wanders freely but still forms a single closed loop of roughly predictable size.

The walk keeps track of two angles: the *reference heading* φ_i, the tangent direction of the reference circle, and a *heading offset* δ_i relative to it. The segment direction is $\theta_i = \varphi_i + \delta_i$. Only the offset is random; the reference heading advances deterministically by $2\pi/N$ per step.

The walk starts at $p_0 = (R, 0)$ with $\delta_0 = 0$, i.e. tangent to the circle. Each segment has length $\ell = 2\pi R/N$, the circumference divided by the number of segments. For $i = 0, 1, ..., N-1$, with progress $t_i = (i+1)/N$:

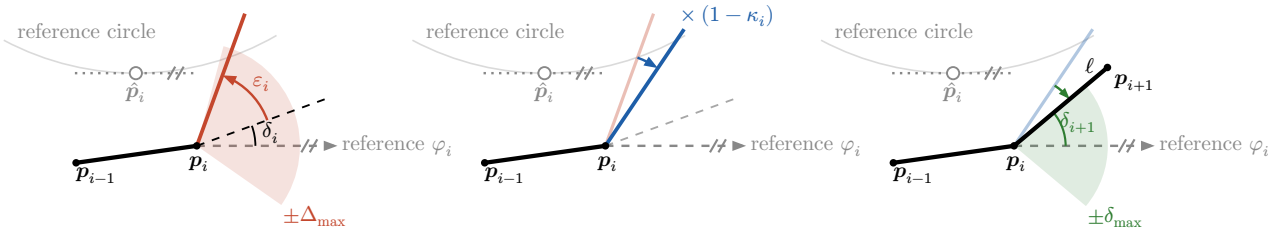
1. We calculate the reference heading, $\varphi_i = 2\pi t_i + \pi/2$, the tangent of the reference circle at $\hat{p}_i = R \cdot (\cos 2\pi t_i, \sin 2\pi t_i)$.
2. We take a random turn (Figure 4.5 a), drawing $\varepsilon_i \sim \mathcal{U}(-\Delta_{\max}, \Delta_{\max})$ and adding it to the offset.
3. As depicted in Figure 4.5 b, we shrink the offset towards zero by a factor $1 - \kappa_i$, where

   $$\kappa_i = \kappa_0 + (1 - \kappa_0) t_i^s.$$

   The restoring strength κ_i starts at κ_0 and ramps up to exactly 1 on the final step. The exponent s controls how late the ramp kicks in.
4. In Figure 4.5 c, we limit the offset from the reference heading to prevent accumulation of large offsets, and advance by length l:

   $$\delta_{i+1} = \mathrm{clip}((1 - \kappa_i)(\delta_i + \varepsilon_i), -\delta_{\max}, \delta_{\max}),$$

   $$p_{i+1} = p_i + \ell(\cos(\varphi_i + \delta_{i+1}), \sin(\varphi_i + \delta_{i+1})).$$



(a) Random turn (b) Mean reversion (c) Clip and step

Figure 4.5: One step of the track-generation walk. **(a)** A random turn $\varepsilon_i \sim \mathcal{U}(-\Delta_{\max}, \Delta_{\max})$ is added to the carried offset δ_i, measured from the reference heading φ_i. **(b)** A fraction κ_i of the accumulated offset is removed, pulling the heading towards the reference φ_i. **(c)** The offset is clipped to $\pm\delta_{\max}$, giving δ_{i+1}, and a step of length ℓ is taken to $\boldsymbol{p}_{i+1}$.

Early on, $\kappa_i \approx \kappa_0$ is small, so the offset behaves like a weakly damped random walk (an AR(1) process) and the track meanders. Near the end, $\kappa_i \to 1$ forces the heading

back onto the reference tangent, so the walk heads towards the start point. Because the integrated heading still deviates slightly from the circle, p_N generally misses p_0 by a small gap. We close it by distributing the gap linearly along the track:

$$p_i \leftarrow p_i + \frac{i}{N}(p_0 - p_N), \quad i = 0, ..., N.$$

The gap is (usually) small, so this correction barely distorts the track.

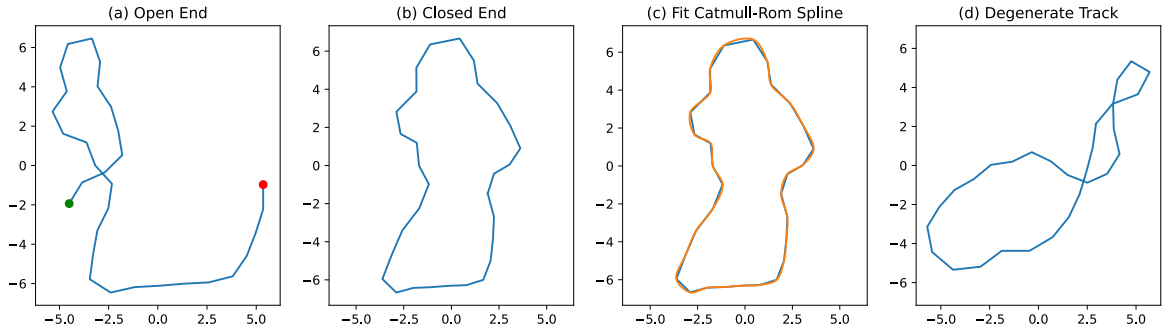


Figure 4.6: (a) Open end of the track before distributing the gap between p_0 (green) and p_N (red) **(b)** Closed track after distributing the gap **(c)** Catmull-Rom spline interpolation of the polyline **(d)** A degenerate track

This method is not immune to generating self-intersecting (degenerate) tracks, but the probability is low enough that we simply discard any degenerate tracks and generate a new one. Empirically we find 50 tracks of 100 000 to be degenerate with our chosen parameters, so a 95%-confidence interval (C.I.) for $P(\text{degenerate track} \mid N = 30, \Delta_{\max} = 80°, \kappa_0 = 0.1, s = 3, \delta_{\max} = 90°)$ is $(0.037\%, 0.065\%)$ assuming a binomial distribution.

4.3.2. Track walls

Adding track walls to the generated centerline spline is a non-trivial problem. A naive approach is to scale the centerline's vertices outwards by a fixed multiple, or provide a fixed offset from the center, but this can create undrivably narrow sections, as visualised in Figure 4.7 (a) and (b) respectively. We therefore generate the walls using the Clipper2 library [48], which implements a polygon offsetting algorithm inspired by Vatti in [49]. The Clipper2 algorithm is robust to self-intersections, but may produce inset "islands" for large inset distances, or for tracks which are close to self-intersecting, as visualised in Figure 4.7 (c).

Empirically, we find that 1350 of the 100 000 tracks produce these islands (C.I. $(1.28\%, 1.42\%)$), so we simply discard any tracks with islands and re-generate them.

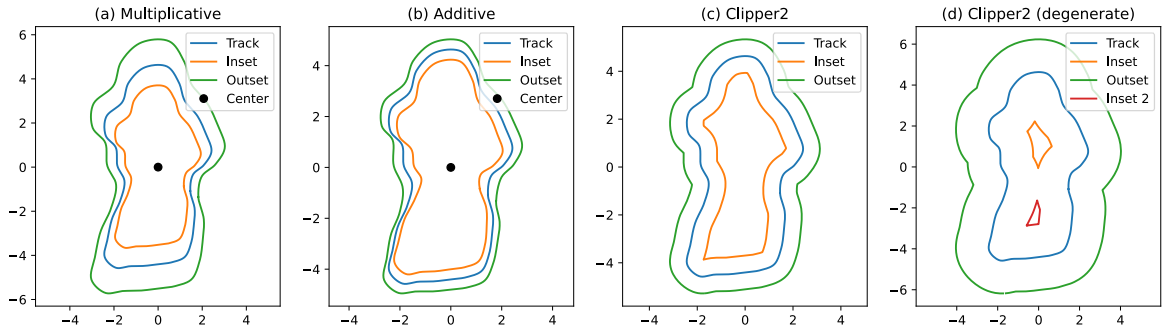


Figure 4.7: Track insets and outsets using different methods

4.3.3. Track Curvature

During trainin, we wish to observe the effect of using track curvature as an auxiliary loss. To this end, we compute the curvature of the track at 16 equally-spaced points along the track ($d \in \{0.5, ..., 8\}$ m) ahead of the car's current position. The curvature is computed using the formula $c_d = \frac{1}{r}$, where r is the radius of the osculating circle at that point on the track. The osculating circle was introduced by Leibniz [50] as cited by [51, p. 419, n. 6]; it is the circle that best approximates a curve at a given point, and its radius can be computed from the first and second derivatives of the track's parametric equations [52, p. 910-911]:

$$\frac{1}{\kappa} = r = \frac{\left(x'(t)^2 + y'(t)^2\right)^{\frac{3}{2}}}{|x'(t)y''(t) - y'(t)x''(t)|} = \frac{|\boldsymbol{r}'(t)|^3}{|\boldsymbol{r}'(t) \times \boldsymbol{r}''(t)|}$$

Chapter 5

The Agent

In this chapter, we describe the architecture of the agent and the auxiliary losses used to improve its performance. Because we are comparing the effect of auxiliary losses, we train multiple agents with the same architecture, but with different auxiliary losses enabled each time.

We shall first formalise the problem of autonomous racing as a partially observable Markov decision process (POMDP) in Section 5.1, then describe the process of reinforcement learning in Section 5.2, describe the architecture of the agent in Section 5.3, [FINISH]

5.1. Partially Observable Markov Decision Processes (POMDPs)

In order to formally describe the problem of autonomous racing, we first introduce the framework of partially observable Markov decision processes (POMDPs). POMDPs are a generalisation of Markov decision processes (MDPs) that account for the fact that an agent may not have access to the full state of the environment. In a POMDP, the agent receives observations that provide partial information about the underlying state, and must make decisions based on this incomplete information.

A POMDPs is defined by the tuple $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{P}, \rho_0, \mathcal{R}, \gamma, \mathcal{Z})$, based on the work by Kaelbling et al. [53]:

At each time step t, the environment is in state $s_t \in \mathcal{S}$, where $\mathcal{S}$ is the state space. This state is a *complete* description of the environment at that time. The first state s_0 is sampled from an initial state distribution ρ_0. The agent does not see s_t directly. Instead, it receives an observation $o_t \in \mathcal{O}$, typically its sensor inputs (e.g. LiDAR, camera frames), drawn from the observation function, $o_t \sim \mathcal{Z}(\cdot \mid s_t)$. The information in s_t that is missing from o_t is called *privileged information*. For example, the car may observe its lidar measurements, but not the road surface conditions directly.

Based on its observation, the agent uses its policy π to choose an action $a_t \in \mathcal{A}$, where $\mathcal{A}$ is the action space. The policy therefore maps an observation to a probability distribution over actions; we write $a_t \sim \pi(\cdot \mid o_t)$[2]. The policy is what we want to find: it describes the agent's behaviour. For simplicity, we let the policy depend only on the current observation; should the policy depend on a history of observations, we would write $\pi(\cdot \mid o_t, o_{t-1}, ...)$. After the agent has taken action a_t, the environment moves to a new state, $s_{t+1} \sim \mathcal{P}(\cdot \mid s_t, a_t)$, where $\mathcal{P}$ is the transition function. By the *Markov property*, the next state s_{t+1} depends only on the current state s_t and action a_t, not on earlier history [16]. Then, the agent receives a scalar reward $r_t = \mathcal{R}(s_t, a_t, s_{t+1})$, where $\mathcal{R}$ is the reward function, together with the next observation o_{t+1}. This interaction loop is illustrated in Figure 5.1.

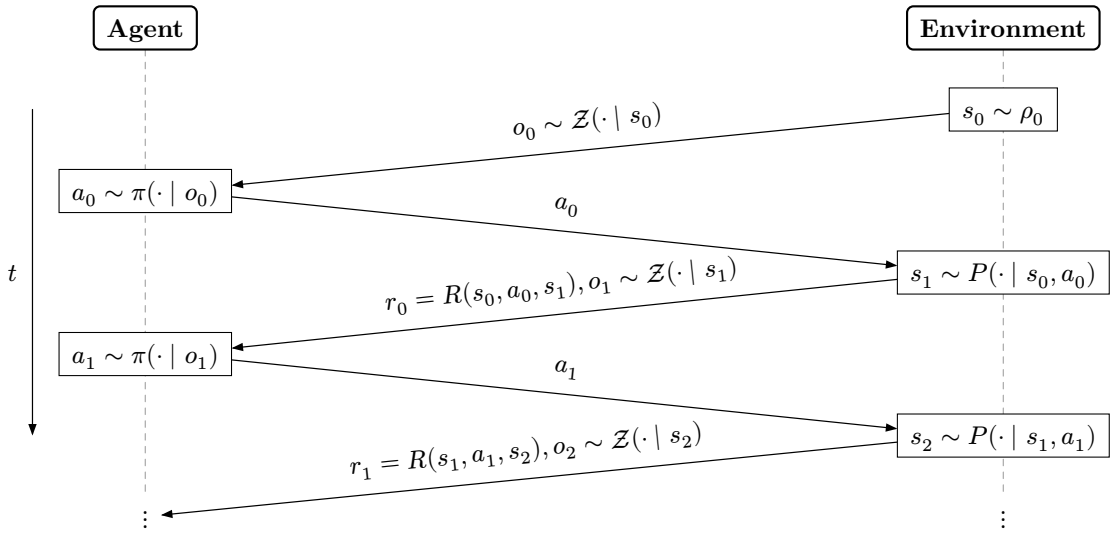


Figure 5.1: Interaction between the agent and the environment over time (t).

The agent's goal is to maximise the *expected discounted return*. The *discounted return* (or simply *return*) from time step t is the discounted sum of all rewards the agent receives from t onward,

$$G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k},$$

where T is the final time step and $\gamma \in [0, 1]$ is the discount factor. A reward received k steps ahead is scaled by γ^k, so later rewards are worth less than immediate ones: when γ is close to 1 the agent is far-sighted, and when γ is close to 0 it is short-sighted. Since G_t depends on random states, observations and actions, it is a random variable, and

[2]Some formulations may define the policy as a mapping from the true state estimate to actions instead, but this is not an end-to-end approach, since there must exist a state estimator in that case.

the agent maximises its expectation, $\mathbb{E}[G_0]$, from the start of the episode. The policy that achieves this maximum is the *optimal policy* π^*.

5.1.1. Value functions

Value functions measure how good it is for the agent to be in a state, or to take an action in that state, and are the basis for the critic in PPO. The state-value function,

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

is the expected discounted return starting in state s and then following policy π thereafter. The action-value function,

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

measures the expected discounted return starting in state s, taking action a, and then following policy π thereafter. Their difference is the advantage function,

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s), \tag{1}$$

which measures how much better action a is than the policy's average action in state s.

5.2. Reinforcement Learning (RL)

In order to find the optimal policy, π^* that maximises the expected discounted return, $J(\theta) = \mathbb{E}_\pi[G_0]$, we use reinforcement learning (RL) as introduced by Sutton and Barto [16]. In RL, the agent learns to maximise its expected discounted return by interacting with the environment and receiving feedback in the form of rewards. The agent updates its policy based on the observed rewards and transitions, gradually improving its performance over time. We use RL instead of supervised learning due to lack of labelled data for the optimal policy.

So-called model-free RL algorithms fall into three broad classes: *Value-based* methods learn the optimal action-value function $Q^*(s, a) = \max_\pi Q^\pi(s, a)$ and act greedily with respect to it, choosing the action with the highest value in the current state: $\pi(s) = \arg\max_{a \in \mathcal{A}} Q^\pi(s, a)$. *Policy-based* methods, on the other hand, learn a parameterised policy π_θ directly by ascending the estimated gradient of $J(\theta)$ with respect to θ, and do not need the Markov property of the observations or an $\arg\max$ over actions. They can also represent stochastic policies, which may be necessary for optimal behaviour in some POMDPs [54] (although there always exists a deterministic optimal policy in fully-observed MDPs). *Actor-critic* methods combine both: a learned value function (the critic) is used to reduce the variance of the policy gradient, and since the critic is

only used in training, it may take privileged information as input as in . We focus on policy-based and actor-critic methods in this report.

5.2.1. Policy gradient theorem

Policy gradient methods are a class of RL training algorithms that learn a policy π_θ with parameters θ directly by maximising the expected return $\mathbb{E}[G_0]$ with respect to the policy parameters θ. Given a trajectory $\tau = (s_0, o_0, a_0, r_0, s_1, o_1, a_1, r_1, ..., s_{T-1}, o_{T-1}, a_{T-1}, r_{T-1}) \sim p_\theta$, where p_θ is the probability of the trajectory when following the policy π_θ, we intend to maximise the expected return with respect to the policy parameters θ:

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[G_0] = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{T} \gamma^t r_t\right] = \int p_\theta(\tau) G(\tau)\, \mathrm{d}\tau$$

Taking the gradient of $J(\theta)$ with respect to θ and applying the log-derivative trick [55], we obtain the policy gradient theorem [16]:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{T-1} \gamma^t \nabla_\theta \log \pi_\theta(a_t \mid o_t) G_t\right]. \tag{2}$$

Equation 2 shows in what direction to update the policy parameters in order to increase the expected return. Of course, we cannot compute the expectation exactly without simulating all possible trajectories, and instead have to rely on estimates such as those detailed in Section 5.2.2 and Section 5.2.3.

5.2.2. REINFORCE

The REINFORCE algorithm for policy learning was introduced by Williams in 1992 in the journal *Machine Learning* [55]. It is one of the simplest policy gradient methods: by sampling N trajectories $\tau^{(i)} \sim \pi_\theta$ of length T_i timesteps each, the policy gradient can be estimated in a Monte-Carlo fashion:

$$\widehat{\nabla_\theta J}(\theta) = \frac{1}{\sum_{i=1}^{N} T_i} \sum_{i=1}^{N} \sum_{t=0}^{T_i - 1} \nabla_\theta \log \pi_\theta\left(a_t^{(i)} \mid o_t^{(i)}\right) \cdot G_t^{(i)}$$

Although REINFORCE is simple to implement, it has several drawbacks, including having high variance in its gradient estimates, which can lead to slow convergence and instability during training [56], and, being an on-policy algorithm, it can only learn from trajectories generated by the current policy, which can limit exploration and learning efficiency (since it cannot train multiple epochs on the same rollout).

5.2.3. Proximal Policy Optimisation (PPO)

PPO is a modern actor-critic algorithm for policy learning introduced in 2017 by researchers at OpenAI [57]. It improves on REINFORCE in two ways. First, being an actor-critic method, it learns a value function $V^\pi(s)$, which is used to compute an advantage estimate $\hat{A}_t$ and so reduces the variance of the policy gradient estimate, $\widehat{\nabla_\theta J}(\theta)$. Second, it optimises a *clipped surrogate objective*, $\mathcal{L}_{\text{PPO}}$, that limits how far each update can move the policy. This prevents large, destructive steps and makes several epochs of updates on the same rollout possible.

Recall that the advantage function in Equation 1 is the difference between the action-value and state-value functions. It therefore measures how much better (positive) or worse (negative) an action is than the policy's average action in that state. In training, $\hat{A}_t = \hat{A}^\pi(s_t, a_t)$ is estimated from observed rewards and the learned value function, $\hat{V}^\pi(s)$, for example with Generalised Advantage Estimation [58]. In PPO, the clipped surrogate objective is given by

$$\mathcal{L}_{\text{actor}} = -\frac{1}{\sum_{i=1}^{N} T_i} \sum_{i=1}^{N} \sum_{t=0}^{T_i-1} \min\Big(\rho_t(\theta) \cdot \hat{A}_t, \text{clip}(\rho_t(\theta), 1-\varepsilon, 1+\varepsilon) \cdot \hat{A}_t\Big)$$

where $\rho_t(\theta) = \frac{\pi_\theta(a_t \mid o_t)}{\pi_{\theta_{\text{old}}}(a_t \mid o_t)}$ is the ratio of the new policy's probability of taking action a_t given observation o_t to the old policy's, and ε is a hyperparameter that controls how much the policy is allowed to change in a single update. The clip function ensures that a policy update that would make $r_t(\theta)$ fall outside of the interval $[1-\varepsilon, 1+\varepsilon]$ is clipped to the nearest endpoint of that interval. This prevents large policy updates that could destabilise training. The min function on the other hand ensures that, while the policy's update is capped when an update makes a good action ($A_t > 0$) more likely, it is not protected from penalties if an update makes a bad action ($A_t < 0$) more likely. These two mechanisms together ensure that the policy is updated in a way that improves performance while avoiding large, destabilising changes that are common in other policy gradient methods [57].

Since PPO is an actor-critic method, it also learns a value function $V^\pi(s)$, which is used to compute an advantage estimate $\hat{A}_t$ and so reduces the variance of the policy gradient estimate, $\widehat{\nabla_\theta J}(\theta)$. The value function is learned by minimising the mean squared error between the predicted value and the observed return. The full PPO loss function is therefore a combination of the clipped surrogate objective and the value function loss, along with an entropy bonus to encourage exploration. The final loss function is given by

$$\mathcal{L}_{\text{PPO}} = \mathcal{L}_{\text{actor}} + w_{\text{value}} \cdot \mathcal{L}_{\text{value}} - w_{\text{entropy}} \cdot \mathcal{H}[\pi_\theta]$$

5.3. Neural Network Architecture

The agent is a feedforward actor-critic network implemented in PyTorch [59], with the TorchRL library [60].

It has no recurrent state, so the stack of past scans is its only source of temporal information, since during testing, a Recurrent Neural Network (RNN) performed poorly in comparison. It consists of a shared *trunk* followed by three independent heads, each of which takes as input the same 64-dimensional feature vector output from the trunk, as visualised in Figure 5.2:

- the *policy head* outputs the parameters of the action distribution,
- the *value head* outputs the state-value estimate $\hat{V}^\pi(s)$, and
- the *auxiliary head* outputs predictions of the privileged targets

The trunk (Figure 5.2) has a convolutional part followed by a dense part. The input is a tensor of shape 50×101 (time steps by rays). Three 1D convolutions with ReLU activations slide along the ray axis and treat the time steps as input channels, so the first layer mixes the whole history at each angular position. They have 32, 16, and 16 output channels respectively; all have the same kernel size of 8 (roughly 21° of FOV). The output is flattened and concatenated with the current steering angle and throttle, and passed through fully connected layers of 128, 72 and 64 units with ReLU activations, followed by layer normalisation of the 64-dimensional output to prevent gradient explosion when the car is under heavy acceleration, pointing the LiDAR above the track walls and resulting in large normalised ray lengths as input (≈ 1).

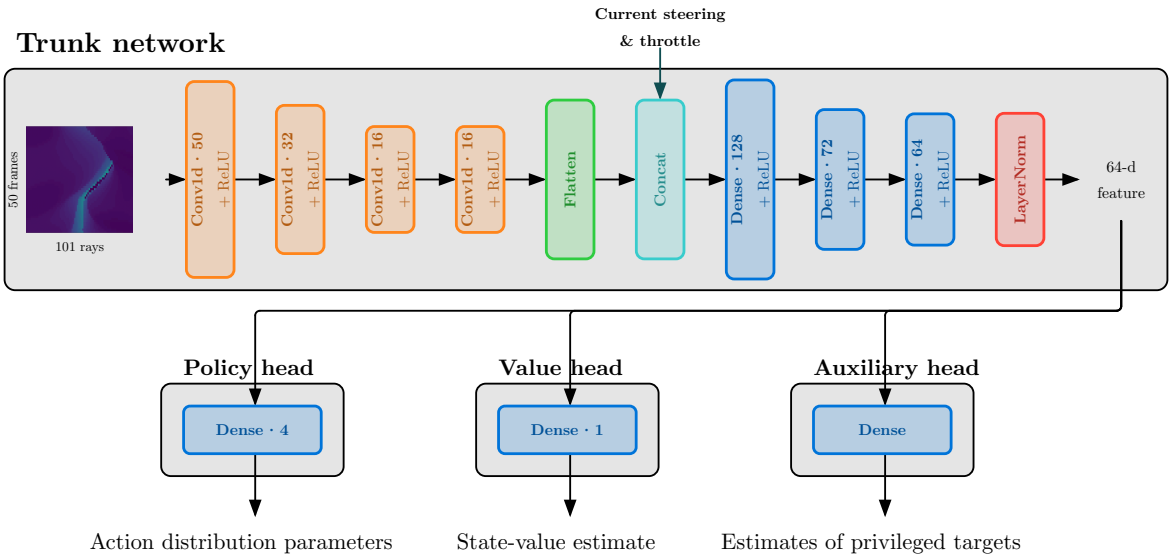


Figure 5.2: The agent's neural network. A 50-frame LiDAR stack passes through four 1D convolutions; the flattened output is concatenated with the current steering and throttle and passed through three dense layers and a layer norm to give a 64-dimensional feature vector. This shared feature vector is then passed to three independent heads: the policy head outputs the parameters of the action distribution, the value head outputs the state-value estimate, and the auxiliary head outputs predictions of privileged targets. The network is drawn at 70% scale to fit on the page.

5.3.1. Action Distributions

The policy head outputs a location μ and a raw scale value x for each of throttle and steering. Each action is sampled independently from a tanh-squashed Gaussian,

$$a = \tanh(u), \quad u \sim \mathcal{N}\left(5 * \tanh\left(\frac{\mu}{5}\right), \sigma^2\right),$$

so that $a \in (-1, 1)$.

The mean is transformed with a tanh to soft-clamp it to the range $(-5, 5)$ using `tanh_loc=True` in the TorchRL `TanhNormal` distribution. This prevents the mean from growing too large, which would push samples into the saturated region of the squashing function and make it difficult for the policy to recover since gradients vanish there. The constant factor of 5 is the heuristically chosen default factor configured in TorchRL [60].

The standard deviation is made positive with a softplus, with a small floor to ensure that the distribution is never degenerate: $\sigma = \text{softplus}(x) + 10^{-3}$.

The policy head's linear layer uses a weight initialisation with gain 0.01 and zero bias, so every output is close to 0 at the start. Hence $\mu \approx 0$, and both actions are centred near the middle of their range.

This is important because the derivative of the squashing tanh function is

$$\frac{\mathrm{d}a}{\mathrm{d}u} = 1 - \tanh^2(u),$$

which goes to 0 as $|u|$ grows. If the head's outputs start large, samples land in the saturated region, the gradient of the policy loss with respect to μ is almost zero, and the policy cannot recover. In early experiments this is exactly what happened: the car locked at full throttle and full steering in either the positive or negative direction.

5.3.2. Auxiliary heads

Using privileged information during training has the potential to improve the agent's performance, as discussed in Section 2.2. We opt to use privileged information in the form of auxiliary losses, which are additional loss terms that encourage the agent to learn useful representations of the environment. These auxiliary losses are computed using privileged information that is not available to the agent during testing, but can be used during training to improve the agent's performance.

Each frame, the environment emits a set of privileged quantities to the trainer, which are used as *auxiliary targets*. The *auxiliary head(s)* predict these targets during the training process. The auxiliary targets we consider are listed in Table 5.1. During training, the auxiliary losses are added to the PPO loss so that their gradients are combined and backpropagated through the network graph:

$$\mathcal{L}_{\text{total}} = \underbrace{\mathcal{L}_{\text{actor}} + w_{\text{value}}\mathcal{L}_V - w_{\text{entropy}}H[\pi_\theta]}_{\mathcal{L}_{\text{PPO}}} + \sum_{k=1}^{K} w^{(k)}\mathcal{L}_{\text{aux}}^{(k)},$$

so that

$$\nabla_\theta \mathcal{L}_{\text{total}} = \nabla_\theta \mathcal{L}_{\text{PPO}} + \sum_{k=1}^{K} w^{(k)} \nabla_\theta \mathcal{L}_{\text{aux}}^{(k)}$$

Table 5.1: The auxiliary targets to be predicted by the agent[3]

Target	Units	Description
Forward velocity v_f	m/s	The component of the agent's velocity in the direction of its forward axis
Lateral velocity v_l	m/s	The component of the agent's velocity in the direction of its lateral axis
Yaw rate ω_y	rad/s	The rate of change of the agent's heading about its vertical axis
Track curvature ahead c_d	1/m	The curvature of the track centerline at d meters ahead of the agent, measured as the reciprocal of the radius of curvature
Per-wheel longitudinal slip κ_x	unitless	The longitudinal slip of wheel $x \in \{\text{FR}, \text{FL}, \text{BR}, \text{BL}\}$
Per-wheel lateral slip $\tan(\alpha_x)$	unitless	The lateral slip of wheel $x \in \{\text{FR}, \text{FL}, \text{BR}, \text{BL}\}$

5.3.3. Choice of Auxiliary Loss Function

The choice of auxiliary loss functions $\mathcal{L}_{\text{aux}}^{(k)}$ has the potential to significantly impact the agent's performance. Since we are predicting continuous quantities, it is natural to use a regression loss function such as Squared Error (SE), Absolute Error (AE), or Huber loss [61], defined respectively as

$$\mathcal{L}_{\text{SE}} = (\hat{y} - y)^2, \quad \mathcal{L}_{\text{AE}} = |\hat{y} - y|, \quad \mathcal{L}_{\text{Huber}}^{\beta}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 \text{ if } |y - \hat{y}| \leq \beta \\ \beta\ |y - \hat{y}| - \frac{1}{2}\beta^2 \text{ otherwise.} \end{cases}$$

Throughout, we use PyTorch's rescaled implementation of the Huber loss (`smooth_l1_loss`), $\frac{\mathcal{L}_{\text{Huber}}^{\beta}(y,\hat{y})}{\beta}$. Dividing by β decouples the threshold from the gradient: the rescaled loss is quadratic with gradient $\delta = y - \hat{y}$ for $|\delta| \leq \beta$, and linear with a constant gradient of magnitude 1 for $|\delta| > \beta$, so that the parameter β sets only the transition point and not the slope of the linear region.

The Huber loss is a robust alternative to the SE loss that is far less sensitive to outliers. It behaves quadratically for small errors and linearly for large ones, with the transition point set by a threshold parameter β. As visualised in Figure 5.3, this means the gradient magnitude is bounded no matter how large the prediction error grows, whereas the L2 gradient grows without limit. Unlike the L1 loss, whose gradient changes

[3]In order to ensure the targets are on a similar scale, the targets are normalised to $\sigma = 1, \mu = 0$ using a rollout generated from a pure PPO policy (i.e. without auxiliary losses/heads) as data to estimate σ and μ.

sign abruptly at zero, the Huber loss is smooth there, so the gradient shrinks gradually as the error approaches zero.

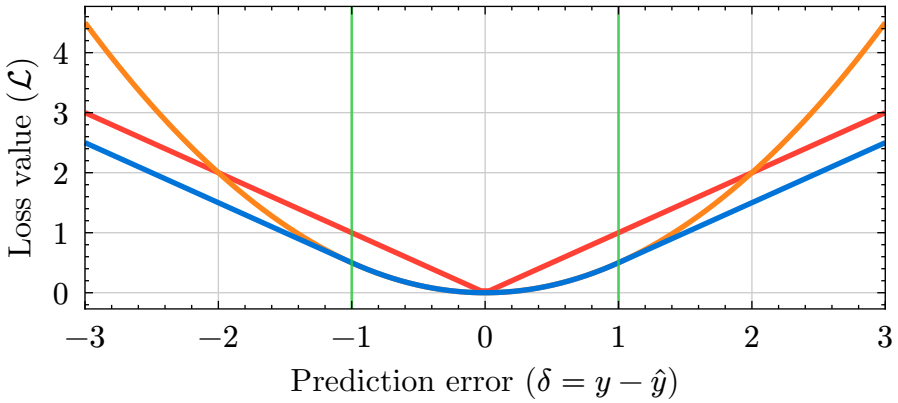


Figure 5.3: Rescaled Huber loss (blue) with the transition point at $\beta = 1$ (green) vs squared error loss (orange) vs absolute error loss (red) for prediction errors δ.

The Huber loss's bounded gradient is particularly useful when the target values have a heavy-tailed, non-normal distribution, since the loss is then far less sensitive to extreme values than the SE loss. This is the case for our auxiliary targets, as visualised in Figure 5.4.

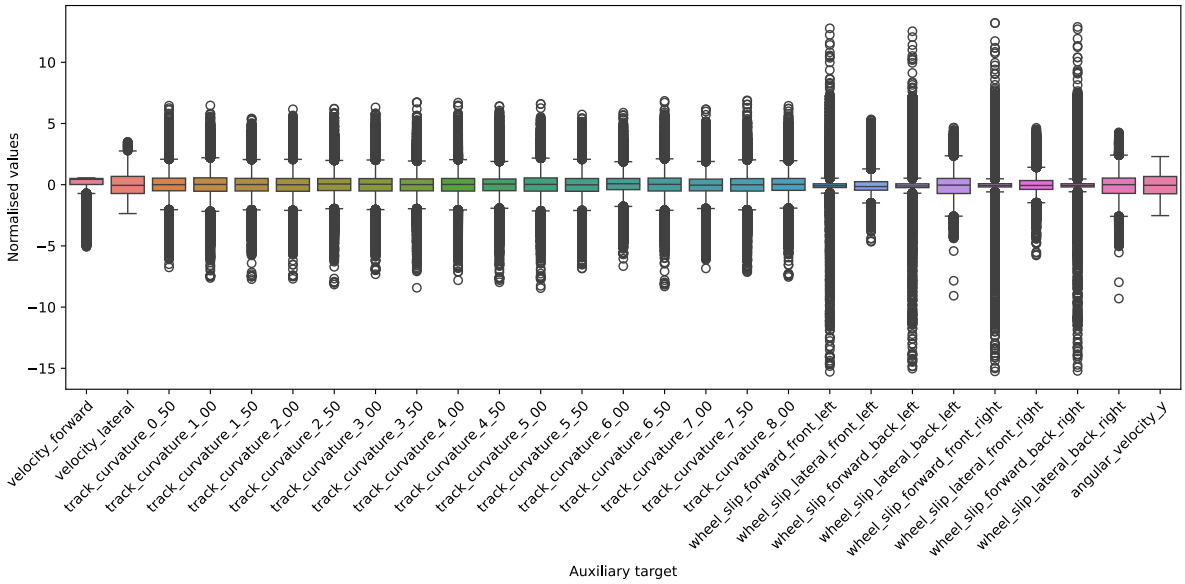


Figure 5.4: Boxplot of the normalised true auxiliary target values collected during a rollout of the baseline policy. It is clear that the targets have so-called "heavy tails".

For this reason, we use the Huber loss for all auxiliary targets, with $\beta = 1$. Since the targets are normalised to zero mean and unit variance (Table 5.1), this makes the loss quadratic for errors within one standard deviation - where most[4] of the fitting takes

[4]Proportions of the data within 1 standard deviation of the mean are listed in Section Appendix C

place - and linear, with a gradient of magnitude 1, beyond it, so that the few extreme targets cannot dominate the gradient.

5.4. Finding the Optimal Weights

The question naturally arises: how should the weights $w^{(k)}$ of the auxiliary losses be set? If they are too small, the auxiliary losses will have little effect on the agent's learning. If they are too large, the agent may use all its capacity to optimise the auxiliary tasks and fail to learn a good policy, or the auxiliary task gradients may conflict with the main task gradients, slowing learning. This is an optimisation problem, where the weights are tuned to achieve the best overall performance. Two options exist: choosing the weights before training (hyperparameter tuning), or adapting the weights during training (adaptive weighting).

5.4.1. Hyperparameter Tuning

Hyperparameter tuning is a common approach to finding the optimal weights for auxiliary losses. It involves running multiple training experiments with different weight configurations and then evaluating the agent's performance, choosing the configuration that yields the best results. This approach is simple to implement, but it can be computationally expensive and time-consuming. It also lacks a guarantee of finding the optimal weights, as the search space can be large and complex.

The simplest tuning method is a grid search, where a discrete set of weight values is defined for each auxiliary loss and all possible combinations are evaluated. However, this approach can be inefficient, especially when the number of auxiliary losses is large, as the number of combinations grows exponentially with the number of weights considered.

Random search [62] is another hyperparameter tuning method, where weight configurations are sampled randomly from a predefined distribution. This approach can be more efficient than grid search, as it allows for exploration of a larger search space without evaluating every possible combination. However, it still requires a significant number of training runs to find the optimal weights.

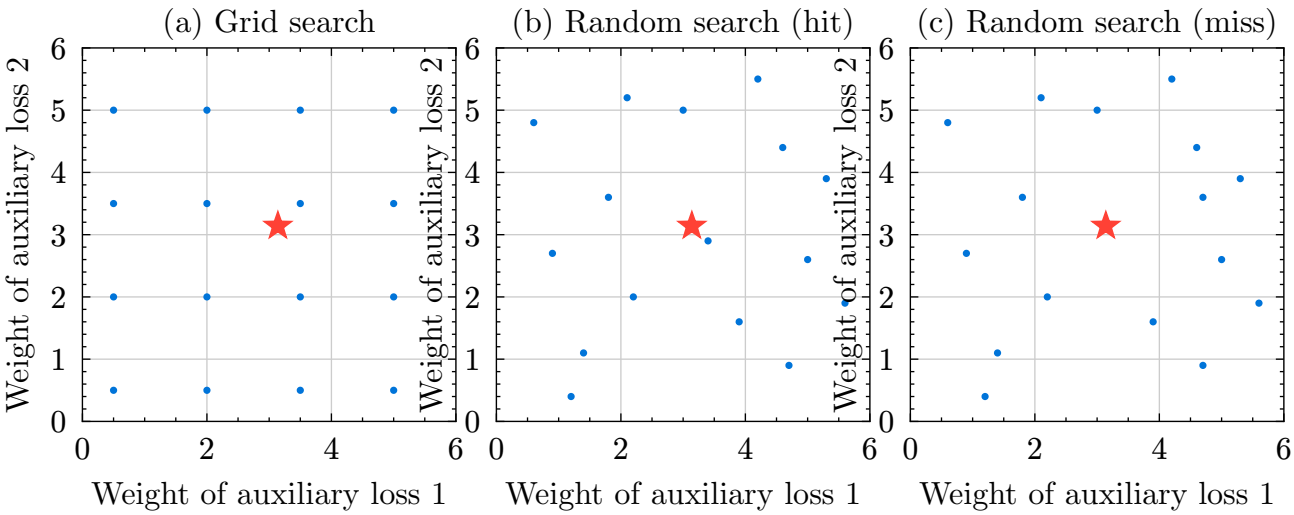


Figure 5.5: Grid and random search over two auxiliary loss weights. The red star indicates the optimal weight configuration. (a) shows that a grid search may miss the optimum, but is guaranteed to find a point within $\frac{\text{grid resolution}}{2}$ of the optimum. (b) shows that random search can find a point very close to the optimum, but (c) shows that it can also miss it entirely.

So-called "smart" hyperparameter tuning methods, such as Bayesian optimisation [63] and the tree-structured Parzen estimator (TPE) [64], build a model of the relationship between the weights and the agent's performance. This model is then used to guide the search for optimal weights, allowing for more efficient exploration of the search space. These methods can be more effective than grid or random search, but they can also be more complex to implement and still require multiple training runs. Methods such as Hyperband [65] and its asynchronous variant ASHA [66] mitigate this cost by giving each trial a small budget and only continuing the best-performing ones. However, such early-stopping methods may prematurely terminate RL trials that perform more exploration early on, despite performing better in the long run, so they are not ideal for our scenario.

5.4.2. Adaptive Weighting

Adaptive weighting is an alternative approach that dynamically adjusts the weights of the auxiliary losses during training based on factors like the magnitude of the gradients or the error of the auxiliary predictions. Various methods have been introduced in the literature, such as GradNorm [67], weights tasks that are learning slower more heavily, and Uncertainty Weighting [68], which uses the model's uncertainty in each task to determine the weights. These methods are useful in multi-task learning scenarios, where the performance of all tasks is important, but they are not ideal for our scenario, where the main task (driving) is the only one that matters. In this case, we want to combine

auxiliary losses in such a way that improves the main task's performance, not to balance the performance of all tasks.

5.4.2.1. PCGrad

PCGrad [69] is a method that modifies the gradients of auxiliary tasks to prevent them from conflicting with the main task's gradients. It does so by projecting the auxiliary task gradients onto the normal plane of the main task's gradient if they conflict. Let $\boldsymbol{g}_{\text{aux}}^{(k)}$ be the gradient of the model parameters for the k-th auxiliary loss and $\boldsymbol{g}_{\text{PPO}}$ be the gradient of the main task loss. If the dot product $\boldsymbol{g}_{\text{aux}}^{(k)} \cdot \boldsymbol{g}_{\text{main}} < 0$, then the auxiliary gradient is projected onto the normal plane of the main task gradient:

$$\boldsymbol{g} = \boldsymbol{g}_{\text{PPO}} + \sum_{k=1}^{K} \begin{cases} \boldsymbol{g}_{\text{aux}}^{(k)} - \frac{\boldsymbol{g}_{\text{aux}}^{(k)} \cdot \boldsymbol{g}_{\text{PPO}}}{\|\boldsymbol{g}_{\text{PPO}}\|^2} \boldsymbol{g}_{\text{PPO}} & \text{if } \boldsymbol{g}_{\text{aux}}^{(k)} \cdot \boldsymbol{g}_{\text{PPO}} < 0 \\ \boldsymbol{g}_{\text{aux}}^{(k)} & \text{otherwise} \end{cases}$$

5.5. Metrics

5.5.1. Progress

Let L be the total length of the track centreline. At any time t, let $d_t \in [0, L)$ be the agent's current arc length from the start of the centerline to the point closest to the agent, and let N_t be the total number of completed laps. We define the agent's *absolute progress* along the track as the total continuous distance travelled from the start:

$$\xi_t = d_t + N_t L$$

The *progress per step* is then simply the change in this absolute position between consecutive timesteps, defined as $\xi_t - \xi_{t-1}$. Because ξ_t tracks the cumulative distance across multiple laps, this raw difference naturally handles crossing the finish line without requiring any wrapping operations or modulo arithmetic.

Using this step-wise progress as a reward signal is approximately equivalent to a potential-based reward shaping function [27]. In this context, the absolute progress ξ_t serves as the potential function $\Phi(s_t)$ of the current state. Because the reward is defined as the difference in potential, $\Phi(s_t) - \gamma\Phi(s_{t-1})$, it heavily incentivises continuous forward movement along the track while theoretically preserving the optimal policy and preventing the agent from exploiting positive-reward cycles.

5.5.2. Crash rates

The *crash rate per episode* is the fraction of total episodes that terminate in a collision rather than reaching the maximum length.

The *crash rate per timestep* is the total number of collisions divided by the total number of timesteps executed. These metrics diverge because episodes terminate early upon a crash; surviving longer before a collision increases the timestep denominator, inherently lowering the per-timestep crash rate.

The *collisions per metre of progress along the centerline* is the total number of crashes divided by the total distance the agent successfully advances along the track centerline. This metric normalizes failures against spatial progression rather than time, isolating driving competence from the agent's velocity.

5.5.3. Reward function

The agent receives a dense reward at each timestep that encourages fast, collision-free progress along the track:

$$r_t = w_p \cdot (\xi_t - \xi_{t-1}) - w_c \cdot c_t$$

The first term rewards forward progress, using the progress per step defined above. The second term penalizes collisions: the indicator variable c_t equals 1 if a collision occurs with an obstacle at time t, and 0 otherwise.

The weights w_p (reward per metre of progress) and w_c (penalty per collision) balance speed against safety based on the risk tolerance of the application. We set $w_p = 1.0$ and $w_c = 1.0$. Because an episode terminates immediately upon collision, there is already a strong implicit penalty for crashing, allowing w_c to remain relatively low.

5.5.4. Area Under the Curve (AUC)

The *Area Under the Curve* (AUC) evaluates sample efficiency and learning stability by calculating the definite integral of a performance metric $y(t)$ (e.g., return or progress per step) over the total training time or timesteps T:

$$\text{AUC} = \int_0^T y(t)\,\mathrm{d}t$$

In practice, evaluation metrics are recorded at discrete interval checkpoints $t_0, t_1, ..., t_N$, and the AUC is computed using the trapezoidal rule:

$$\text{AUC} \approx \sum_{i=1}^{N} \frac{y(t_{i-1}) + y(t_i)}{2}(t_i - t_{i-1})$$

A higher AUC indicates faster learning and superior performance throughout the entire training process rather than merely achieving high final performance.

5.6. Trainer Performance Tuning

Training was performed on a single machine with an AMD Ryzen Threadripper PRO 3995WX CPU (64 cores, 128 hardware threads) and 128 GB of RAM. To ensure experiments finish in a practical time, we first measured how best to use this hardware, aiming to maximise *aggregate* throughput: environment timesteps collected and learned from per second, summed across all models training simultaneously.

We ran a grid search over the parameters in Table 5.2: concurrent trainers (2, 4, 8, 12), Python threads per trainer (8, 16) and parallel environments per trainer (4, 8, 10, 16). For each configuration, `concurrent_trainers` identical processes were launched at once, each running five PPO iterations (a 256-frame rollout plus four epochs of minibatch updates each iteration) with checkpointing and TensorBoard logging disabled. Configurations with more than 8 trainers and 16 environments (128 or more concurrent Unity instances) exhausted memory and were excluded, leaving 28 single-run configurations. Throughput per trainer was computed from the mean wall-clock time per iteration:

$$\text{throughput} = \frac{\text{rollout length} \times \text{parallel envs} \times \text{areas per env}}{\text{seconds per iteration}}$$

We report the throughput per trainer, showing how much each model slows as the machine gets busier, and summed across trainers, showing where adding trainers stops paying off.

As shown in Section Appendix E, total throughput peaks at 8 trainers (4172 timesteps/s with 8 environments and 16 threads) and falls at 12 (at most 3853 timesteps/s), while per-trainer throughput roughly halves from 2 to 8 trainers. The best settings use $8 \times 16 = 128$ python threads in total, matching the hardware thread count; at 12 trainers the 16-thread configurations are oversubscribed and fall behind the 8-thread ones. We therefore chose (`concurrent_trainers`, `num_inst`, `python_threads`) = (8, 8, 16), though its margin over the next-best configurations (4020 and 3962 timesteps/s) is within likely run-to-run variation. Using `num_areas` $\in \{8, 12\}$ per environment in this configuration did not help (3844 and 3999 timesteps/s).

By default, Unity spawns `n_cpus` garbage-collector threads; we set this to 8 with the `-gc-helper-count` launch argument for all runs.

Table 5.2: Parameters varied in the training performance grid search.

Parameter	Values considered	Description
`concurrent_trainers`	$\{2, 4, 8, 12\}$	Number of independent models training simultaneously, each in its own process.
`python_threads`	$\{8, 16\}$	Threads available to each trainer's PyTorch, OpenMP, and OpenBLAS thread pools.
`num_inst`	$\{4, 8, 10, 16\}$	Number of Unity environment instances per trainer. Each contains 6 training areas, so each instance steps 6 agents in parallel.

Chapter 6

Experiments & results

6.1. Baseline Model Performance

The baseline model trained with the PPO loss and no auxiliary targets performed well in its primary task. It achieved a crash rate per meter of progress of [Amount] and a reward AUC of [Amount].

To establish what the baseline represents without auxiliary supervision, linear probes were fit to the output of the final trunk layer (fc3) to predict each auxiliary target. Each probe is an OLS regression model trained on activations from a ~35-minute rollout (52429 timesteps) and evaluated on a separate ~9-minute rollout (13107 timesteps). Figure 6.1 reports out-of-sample R^2 at initialisation and after 600 PPO iterations, with error bars showing the standard deviation between each different trained agent model.

Two distinct patterns emerge. Quantities describing the vehicle's instantaneous dynamics (yaw rate, lateral velocity and lateral wheel slip) are already linearly decodable at initialisation ($R^2 \approx 0.8$–0.9). Since the network is untrained, this reflects information that is close to linear in the observation and survives a series of random LeCun-initialised projections, rather than a learned representation. A similar effect was observed in [70] when investigating the same phenomenon in toy MNIST models. Training does not improve the R^2 for these quantities, and decodability declines modestly (e.g. rear lateral slip falls from $R^2 = \sim 0.67$ to ~0.50), suggesting that PPO discards some of this information.

Figure 6.2 visualises the trend over time: as the model trains, the decodability of the instantaneous dynamics quantities declines, while the decodability of the close-ahead ($\leq 3m$) track curvature improves almost monotonically (highlighted in white boxes). This suggests that these quantities may prove to be useful auxiliary targets, as the auxiliary losses will encourage the model to learn their representations earlier. The model does not significantly learn to represent the further-ahead curvature (4.0m - 8.0m), which presents itself as another candidate for an auxiliary target.

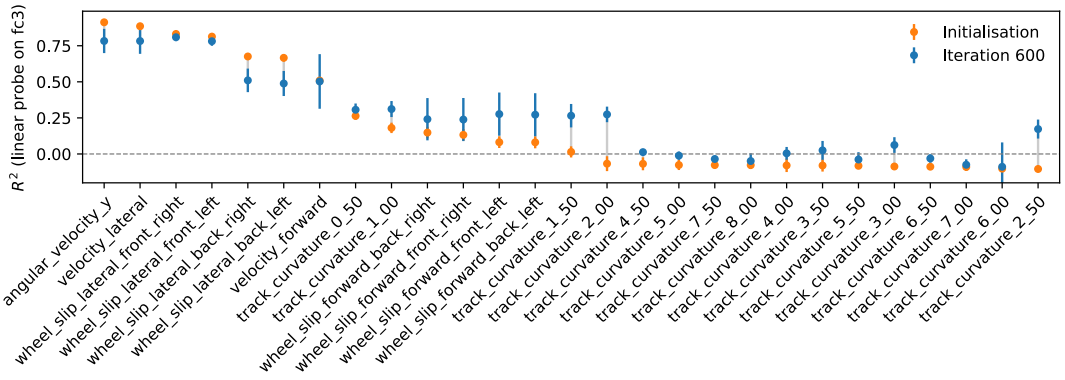


Figure 6.1: Out-of-sample R^2 scores of linear regression probing of the final layer of the trunk on the baseline model.

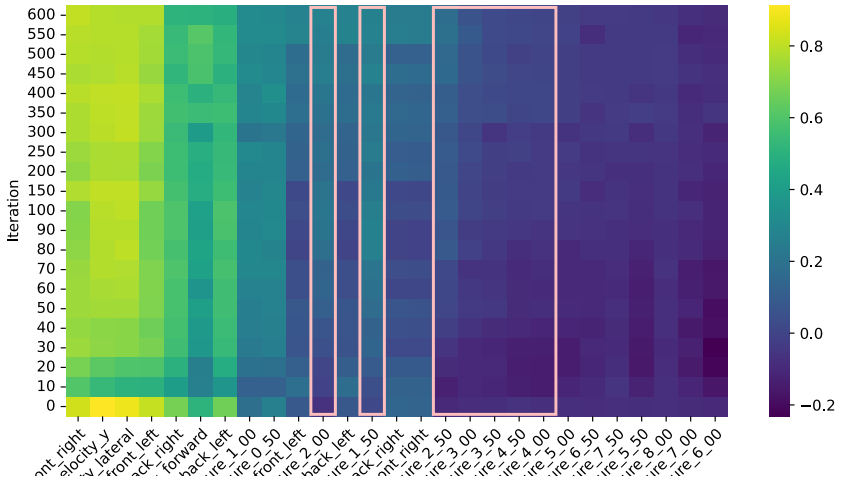


Figure 6.2: Mean out-of-sample R^2 scores of linear regression probing of the final layer of the trunk on the baseline model, showing how much each quantity is linearly decodable at each iteration of training. Note that the y-axis is not evenly spaced.

6.2. Learned Features After Auxiliary Learning

Consider the arbitrary network in Figure 6.3. Once trained, each layer l maps an input $\boldsymbol{x}$ to an intermediate representation $\boldsymbol{h}^{(l)} = f_l(\boldsymbol{x}; \theta)$, which becomes the input to the following layer. We would like to know what information these representations encode, and in particular whether they encode privileged state variables (e.g. velocity, slip, upcoming curvature).

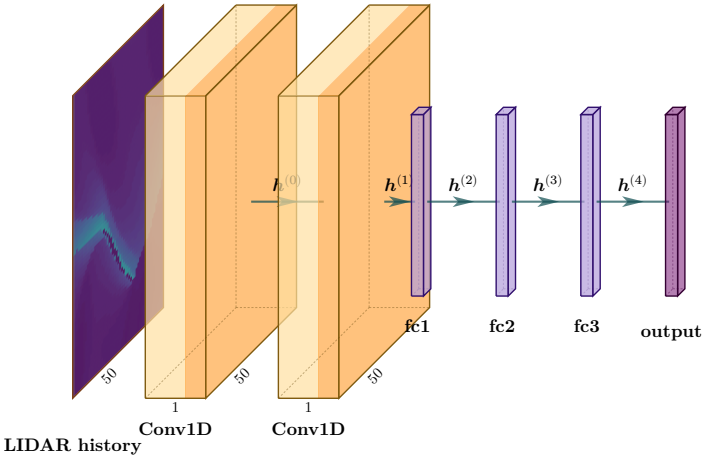


Figure 6.3: Example network architecture. The first two layers are convolutional layers, taking stacked LiDAR frames as input. The following 4 layers are fully connected layers with ReLU activations.

Alain and Bengio [70] introduced *linear probes* for this purpose. The main network's weights are frozen, and a separate linear classifier (the probe) is trained in a supervised manner on the representations $\boldsymbol{h}^{(l)}$ of each layer to predict the desired target. Since the probe cannot change the representation, its test accuracy measures how much class information is *linearly* available at that layer. Anand et al. [71] extended this idea to reinforcement learning, probing an encoder's representations for ground-truth state variables of Atari games (read from the console's RAM).

The choice of a *linear* probe is deliberate. A sufficiently expressive probe (e.g. a deep MLP) may compute the target from almost any input, including the raw network input, so high probe accuracy would say more about the probe than about the representation [72]. Restricting the probe to a linear map means that it can only succeed if the network has already done the work of encoding the target along some direction in the representation space.

Many of the privileged variables of interest are continuous rather than categorical, so we use a linear *regression* probe instead of a classifier, as done by McGrath et al. [73] to probe AlphaZero for chess concepts and by Gurnee and Tegmark [74] to probe language models for spatial and temporal encodings.

The model is now trained with the same reward function (Section 5.5.3) as the baseline model, but with the addition of one or more auxiliary heads (and their associated auxiliary losses).

6.2.1. Forward velocity, lateral velocity, and yaw rate

6.2.2. Track curvature

The close-ahead ($\leq 3m$) track curvature auxiliary loss was extremely effective in improving the decodability of the trunk's final layer. It improved the out-of-sample R^2 scores in close-ahead curvature quantities by between +35 to +49 percentage points, and even improved the decodability of unsupervised quantities such as the 3.5m-ahead curvature and 4.0m-ahead curvature by between +5 to +10 percentage points on average.

To a lesser extent, the medium-ahead (between 3m- and 6m-ahead) curvature auxiliary loss improved the decodability of its supervised quantities, resulting in a +7 to +26 percentage point improvement in out-of-sample R^2 scores.

However, far ahead curvature failed to improve the decodability significantly, with small gains of up to +4 percentage points, and all groups of auxiliary losses decreased performance on the forward slips of wheels. [WHY?]

6.2.3. Wheel slips

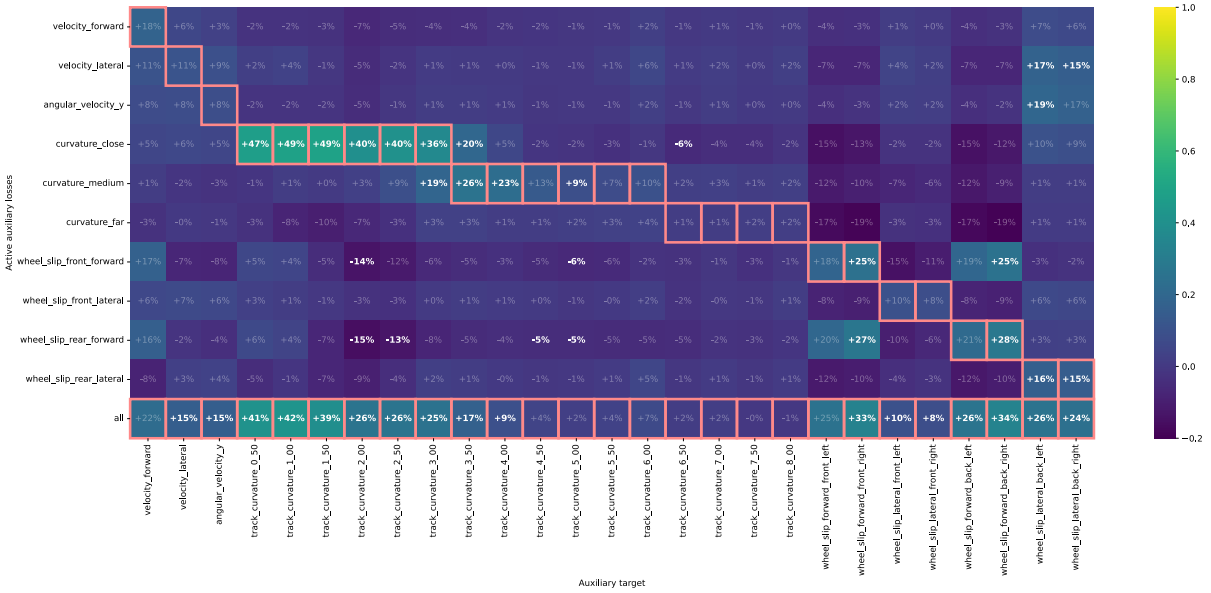


Figure 6.4: Mean out-of-sample R^2 scores of linear regression probing of the final layer of the trunk on models trained with different auxiliary heads enabled. Bold = C.I. excludes 0, corrected for false discovery rate using the Benjamini-Hochberg procedure [75]

6.3. Agent Performance After Auxiliary Learning

In line with the findings from Section 6.2, the auxiliary head that was most effective in improving the agent's performance was the close-ahead curvature head. It resulted in a measurably higher reward AUC compared to the baseline model, as visualised in Figure 6.5. Note that because the model restarts after a crash, the reward AUC is only slightly influenced by the crash penalty (an average of 2.71 reward is generated per second vs -1 per crash). It is therefore important to monitor the performance of the agent in terms of each metric individually, as visualised in Figure 6.6, where the metrics (y-axis) are averaged over the last 50 training iterations and compared to the baseline model's mean. We find that the curvature close model achieves both higher racing performance, as well as safety: it achieves a +12.5% higher mean progress per timestep along with a -38% collision rate per metre of progress at an auxiliary loss weight of $\omega_{\text{aux}} = 1.0$. Similar, though less pronouncd results are achieved by the front wheel forward slip head, which is, in fact, safer (-)

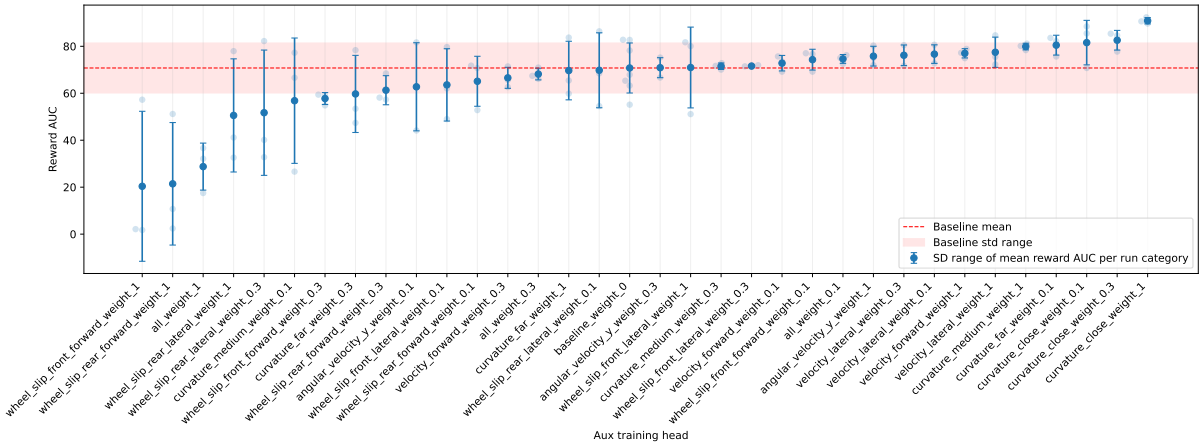


Figure 6.5: Reward AUC of the models trained with different auxiliary losses and loss weights over their 600 training iterations, compared to the baseline model

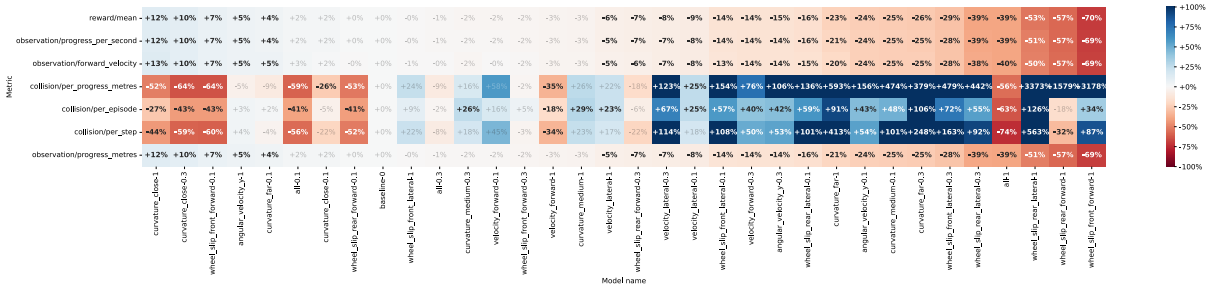


Figure 6.6: Mean of the last 50 training iterations of each model's metrics (mean reward, mean progress per timestep, mean forward velocity, mean number of collisions per metre of progress, mean number of collisions per episode, and mean number of collisions per timestep) vs the baseline model. Bold = statistically significant ($\alpha = 5\%$). No false discovery rate correction. [75]